TUI

React·Ink 터미널 UI

입력 이벤트에서 reconciler, Yoga, ANSI, scrollback까지

분석 대상
v2.1.88 reconstructed source

고정 리비전
2ca5ddabfed5

독자
frontend·CLI 엔지니어

사용 방식
renderer · interaction 분석

> 교육·연구용 파생 분석입니다. Anthropic의 공식 소스·공식 아키텍처 문서가 아닙니다.

“React 앱”이라는 말의 정확한 범위

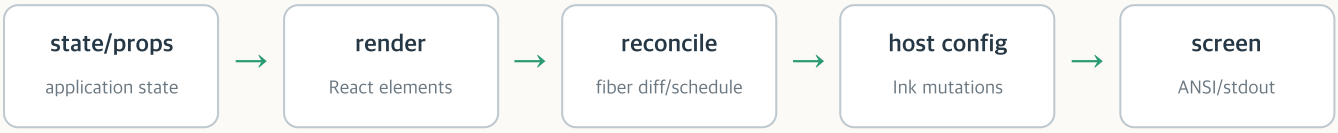
interactive UI는 React component/hook 모델을 사용하지만 agent loop, tool runtime, provider adapter까지 React가 소유하는 것은 아니다.

구역	주요 기술	React의 역할
Prompt/UI	TSX, hooks, Ink primitives	상태 → element tree
Renderer	react-reconciler, custom host config	tree diff/commit coordination
Terminal host	Yoga, ANSI, stdout, cursor	React 밖의 출력 adapter
Agent core	async generator, typed messages	직접 의존하지 않아야 할 이상적 core
Tool UI	ReactNode rendering methods	현재 계약에서는 일부 결합

근거 위치 참고 영상 · src/Tool.ts

reconciler와 renderer를 분리해서 생각하기

React는 element tree의 이전/다음 상태를 비교하고 commit할 변경을 계산한다. host renderer는 그 변경을 DOM 대신 terminal node에 적용한다.



보정

"reconciler = 단순 tree diff 알고리즘"은 입문 설명으로 유용하지만, 실제 React는 scheduling, effects, priorities, host config commit을 포함한다.

근거 위치 React render/commit source · react-reconciler package

DOM이 없는 React renderer

Ink는 component 기반 CLI UI를 제공하고 Yoga로 terminal Flexbox layout을 계산한다. 이 저장소에는 Ink 계열 구현이 `src/ink` 아래 복원되어 있다.

ELEMENT

Box/Text/Static

DOM tag 대신 terminal host primitive를 사용한다.

LAYOUT

Yoga

문자 cell 공간에서 flex layout을 계산한다.

PAINT

ANSI

색·커서 이동·clear sequence를 stdout에 쓴다.

INPUT

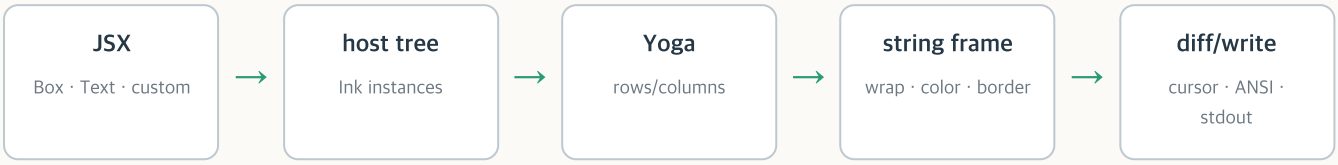
useInput

stdin byte/key를 component callback으로 변환한다.

근거 위치 Ink official repository · src/ink/reconciler.ts · src/ink/render-to-screen.ts

JSX가 터미널 글자가 되기까지

브라우저와 달리 terminal은 pixel DOM이 아니라 character grid와 cursor를 가진 stateful output device다.

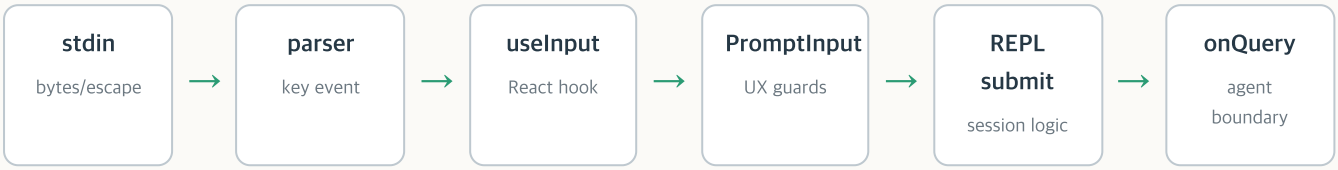


같은 React 상태 모델을 쓰더라도 terminal resize, wide grapheme, scrollback, alternate screen, piped stdout 같은 host constraint가 UI 구조를 바꾼다.

근거 위치 src/ink/layout · src/ink/render-node-to-output.ts · src/ink/log-update.ts

Enter는 단일 callback이 아니라 책임 사슬

raw stdin에서 agent query까지 각 계층이 다른 결정을 소유한다.



책임 분리

low-level input은 key semantics, PromptInput은 선택·추천·빈 입력 같은 UX, REPL은 history·remote·slash/session, onQuery는 agent turn을 소유한다.

근거 위치 src/ink/hooks/use-input.ts · src/components/PromptInput/PromptInput.tsx · src/screens/REPL.tsx

early return은 interaction state machine

Enter가 항상 “모델에게 전송”을 뜻하지 않는다. selection confirm, suggestion accept, agent routing, empty input 등 mode별 의미가 먼저 평가된다.

guard 예	Enter의 의미	return 이유
footer/overlay selection	현재 항목 선택	query를 시작하면 안 됨
agent selection	대상 결정	입력을 agent prompt로 오해 방지
suggestion visible	추천 수락	완성 전송 방지
empty without attachment	no-op	빈 turn 방지
active teammate context	delegate/direct message	leader query와 분리
normal input	parent submit callback	모든 guard 통과

> **코드 표현**
>
> 일곱 개 if를 “복잡한 조건문”으로만 보지 말고, 우선순위가 있는 finite-state transition table로 재작성해 본다.

UI와 agent core가 만나는 seam

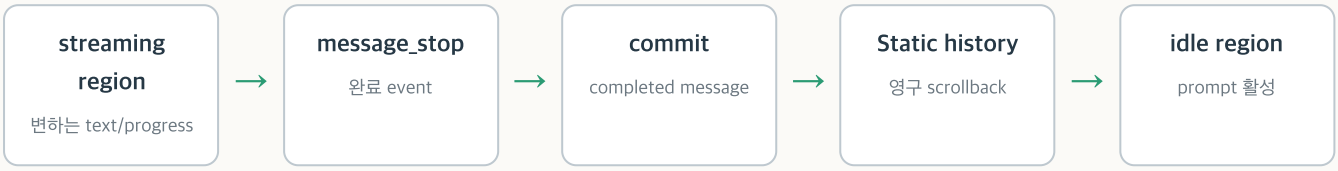
REPL의 onQuery는 UI loading state, abort controller, message accumulation, stream event handling과 query 호출을 조율한다.

UI 쪽	경계	core 쪽
input clear, loading, spinner	`onQuery`	query params 구성
message state append	stream consumer	async generator yield
interrupt key	AbortController	provider/tool signal
permission dialog	canUseTool callback	tool execution gate
turn complete UI	terminal/finally	Terminal reason

근거 위치 src/screens/REPL.tsx

scrollback 보존이 component 구조를 바꾼다

terminal transcript의 과거 부분은 다시 그리지 않는 Static 영역으로 이동하고, 현재 streaming/spinner/input 만 동적으로 갱신한다.



왜 중요한가

매 frame 전체 transcript를 cursor overwrite하면 사용자가 위로 스크롤한 history가 흔들리거나 사라질 수 있다. host의 scrollback semantics가 component architecture를 결정한다.

근거 위치 Ink Static documentation · src/components/Messages.tsx

한 chunk가 화면까지 가는 경로

provider event는 query generator에서 yield되고, REPL consumer가 message/state를 갱신하며, React/Ink가 최소 host 변경을 계산한다.



이 흐름은 “API가 React를 호출한다”가 아니다. async event consumer가 application state를 바꾸고 React가 그 상태의 projection을 그린다.

근거 위치 src/screens/REPL.tsx · src/query.ts

자동 배칭을 16ms 타이머로 단정하지 않기

참고 영상은 빠른 SSE chunk를 React가 묶어 flicker를 줄인다는 직관을 제공한다. 다만 정확한 timing은 React scheduler, Ink renderer, stdout flush 구현과 버전에 달려 있다.

확인 가능

state update batching

여러 update가 하나의 render/commit으로 합쳐질 수 있다.

미확인

고정 16ms

항상 60fps cadence라는 보장은 소스/측정 없이 단정하지 않는다.

HOST COST

stdout write

React render보다 ANSI diff와 terminal emulator가 병목일 수 있다.

측정

frame trace

chunk time, render count, bytes written을 함께 기록한다.

교육 포인트

좋은 설명 모델은 원리를 잡게 하지만, 성능 수치·스케줄 보장으로 확장될 때는 구현과 계측이 필요하다.

interactive surface의 상태 복잡도

호출 빈도는 품질 점수가 아니라, 어떤 React 메커니즘을 우선 학습하고 어디를 더 세밀하게 추적할지 정하는 탐색 지도다.



빈도는 “hooks 남용” 판정이 아니다. UI 상태가 큰 REPL/PromptInput에 집중되어 있는지, custom hook이 lifecycle과 정책을 격리하는지 파일별로 본다.

왜 renderer 구현까지 저장소에 있을까

src/ink에는 reconciler, screen, focus, selection, hit-test, terminal parser, alternate screen 등 host-specific 기능이 넓게 존재한다.

하위 영역	책임	학습 질문
reconciler/renderer	React host mutations	어떤 node operation을 지원하나?
layout	Yoga adapter	cell width와 flex size가 어떻게 만나는가?
termio/parser	키·OSC·escape parse	부분 sequence와 timeout은?
screen/frame/output	virtual screen과 diff	최소 write 전략은?
selection/hit-test	mouse/text selection	좌표와 wide char를 어떻게 처리?
focus/input hooks	active component routing	중첩 overlay의 우선순위는?

근거 위치 src/ink tree

컴포넌트 재사용보다 변경 격리가 우선

TextInput, PromptInput, REPL을 나누는 주된 이유는 재사용뿐 아니라 변경 이유를 다르게 유지하기 위해서다.

구성	변경 이유	소유하지 않아야 할 것
Text/input primitive	key editing·cursor·paste	agent query·session history
PromptInput	completion·suggestion·mode UX	provider API·tool loop
REPL	session lifecycle·messages·remote	low-level escape parsing
onQuery consumer	stream→app projection	tool implementation
query core	agent turn state machine	terminal rendering

현실의 타협

Tool 계약이 render methods를 포함하므로 core/UI 경계는 완전히 깨끗하지 않다. 보고서는 이상화하지 않고 이 결합을 학습 대상으로 남긴다.

TUI 성능은 render 횟수만의 문제가 아니다

성능 병목을 네 층으로 나누면 최적화가 선명해진다.

층	측정	개선 후보
event ingestion	chunk/s, queue depth	coalesce, backpressure
React	render/commit count, component time	state locality, memoization
layout/paint	Yoga time, frame bytes	Static, narrower diff
terminal	write latency, emulator CPU	batch writes, alternate screen choice
history	message count, retained tree	windowing, compact representation

최적화 규칙

체감 latency, dropped update, scrollback correctness를 함께 측정한다. render count만 줄이면 progress observability가 나빠질 수 있다.

UI를 세 층으로 테스트한다

terminal snapshot 하나에 모든 의미를 맡기면 fragile하다.

1. **pure transition** — key/mode/input으로 guard 결과를 table test한다.
2. **component render** — state별 element/output을 Ink test renderer로 확인한다.
3. **host integration** — pseudo-TTY에서 ANSI, resize, Ctrl+C, scrollback을 확인한다.
4. **cross-boundary** — mock async generator로 streaming→completed 전이를 검증한다.
5. **accessibility/operability** — color 없이도 permission/error가 구분되는지 본다.

실습 · frame probe

10ms마다 100개 chunk를 보내고 실제 stdout write 횟수·byte·최종 frame을 측정한다.

완료 증거: 원시 trace + batching에 대한 결론/한계

사용자 입력 한 번을 timestamp로 재구성

영상의 설명을 소스와 측정으로 검증하는 실습이다.

시점	기록할 사건	증거
T0	Enter key parsed	useInput callback
T1	UX guard 통과	selected branch
T2	local user message render	state update/frame
T3	onQuery 시작	query checkpoint
T4	first model delta	requestId/event
T5…n	stream text updates	render/write trace
Tn+1	message stop	Static history commit
Tn+2	input re-enabled	idle state

비교 과제

local echo(T2)가 first token(T4)보다 앞서야 하는 이유를 사용자 경험과 causal ordering 관점에서 설명한다.

터미널 UI를 이해했다는 기준

React를 쓴 사실보다 어떤 host constraint를 어떤 추상화로 격리했는지 말할 수 있어야 한다.

CORE

Reconciler

state에서 변화할 host tree를 계산한다.

ADAPTER

Ink

terminal node, layout, input, output을 제공한다.

HOST

TTY

cursor·ANSI·scrollback 제약이 구조를 결정한다.

BOUNDARY

onQuery

UI projection과 agent event stream을 연결한다.

다음 권

05권은 화면 뒤에서 tool effect를 제한하고 외부 capability를 붙이는 권한·hook·MCP·plugin 구조를 다룬다.